

CSCI 4131 – Internet Programming Assignment 6

Due Date: Friday, May 2nd, 11:59 PM

Late Due Date: Monday, May 5th, 11:59 PM

1. Description

1.1. Purpose

This assignment will have you finish up implementing CRUD operations for your schedule tracking application. In Homework 5, you created the schedule table, implemented C (Create), R (Read), and D (Delete), but now you'll implement editing (Updating) of scheduled events. This specification is 5 pages in length.

1.2 High-Level Overview

This assignment is relatively small, and you will add functionality to your previous homework (Homework 5). Before starting this assignment, you should make sure you're comfortable with querying the database and that your implementation of C, R, and D operations from Homework 5 is working.

Please view the video posted with this write-up for a demonstration of the functionality you will be adding to your Programming Homework 5 solution for this assignment.

2. Preparation

2.1 Provided Files

No additional files are provided for this assignment. You will primarily be working from your Homework 5 submission.

2.2 File Structure

For this assignment, we're requiring the following file structure in place: (see next page)

```
<Your UMN x.500 ID> (folder/directory)
  server.js
  static (folder/directory)
    html (folder/directory)
      All your html files should be stored here -
    img (folder/directory)
      The img folder should contain all the local
      images you use in your HTML Pages
    js (folder/directory)
      The js folder/directory should contain all
      the external JavaScript
      you use in your HTML Pages
    css (folder/directory)
      The css folder/directory should contain all the
      external css files you use in your HTML Pages
  views (folder/directory)
    This should have your editForm.pug file
```

Alongside the previous directories you've made, you'll also now want a views directory, which will contain a single file named editForm.pug

When you submit, make sure you do not submit the node_modules directory. You will lose a significant number of points if you do.

2.3 New Files

You are responsible for creating (and using) the following new file in your solution:

- editForm.pug

3. Functionality

3.1 Add an "Edit" button

You should update your schedule.html page to now also include an "Edit" button for each schedule event. This "Edit" button should be next to the "Delete" button,

and when clicked, send a request to `GET /editSchedule/{EventId}` from the server. Note that `editSchedule` is not a file; the server will need to do some additional work with the `EventId` parameter to return the HTML that we want. More specifics are discussed below.

3.2 Dynamic Form Generation (Server-Side)

When the server gets a request to `GET /editSchedule/{EventId}`, it will need to do the following tasks:

1. Validate that an element with a matching `EventId` exists in the database. If it does not, return a 404 response.
2. If your program successfully retrieves the element:
 - i. Utilize the `editForm.pug` template and the information retrieved from the database to dynamically generate the `editForm` with values pre-filled to match the contents of the database entry.
 - ii. Return the rendered HTML back to the client so that they can submit the form.

In practice, this means that the client can click the “Edit” button for a row, and immediately get a form that has the information of that row automatically filled in to the form without any additional client-side JavaScript.

3.3 Form Submission

When the user submits the edited form, a request should be sent to `POST /updateSchedule/{EventId}`. This request can also be a PATCH request if desired, but should contain the information that the user submitted with the form back to the server. When submitted, the server should then do the following:

1. Validate that an element with that `EventID` exists; if it does not, return a 404 response.
2. Update the matching row in the database with the new values. Only that row should be updated; no new rows should be created, nor should there be any deleted rows.
3. If the update fails (say due to invalid types or some other validation issue), return a generic error message back to the client. You may give additional information like what specific error was raised, but for now, we just want the client to know “something went wrong”.
4. If the update is successful, route the client back to the main `schedule.html` page. With the client-side JavaScript written in Homework 5, you should be able to see the updated row with its new values in your schedule page.

6. Submission Instructions

You must submit your server with all required files (see 2.2 File Structure), **but specifically no node_modules**.

There will be no auto-grader validation for this assignment beyond looking for node_modules. If node_modules are present, we will dock a not-insignificant number of points.

7. Evaluation Criteria

The HTTP server you submit will be graded out of 100 possible points. Your submission will be graded on the following items:

- Schedule page table updates (15 points)
 - An "Edit" button is properly placed for each element
 - Each edit button corresponds to a unique singular row (clicking the edit button for the last row should not route me to the edit page of the first row)
 - The edit button is generated using client-side JavaScript
- Server updates for the Edit Button - Database and routing logic (25 points)
 - The server should return a 404 if the requested element does not exist in the database
 - If the element does exist, the server should return an edit page for that element with fields pre-populated
 - The specific request supported by the server should be to `GET /editSchedule/{EventId}`
- Server updates for the Edit Button - Pug (15 points)
 - The rendered edit form should be in editForm.pug
 - The server should use Pug templating with the event returned from the database to render the Pug template on the server-side before returning the resultant HTML to the client.
- Server handling updates (35 points)
 - When the form is submitted, a POST request is sent to `POST /updateSchedule/{EventId}` (PATCH is acceptable as well)
 - The server should then validate that the entry exists and gracefully return a 404 response if it does not
 - The server should update the matching entry with the new values provided in the POST request. If validation fails, the server should gracefully return a failure page. If validation succeeds and the database is properly updated, the server should reroute the user to the schedule page

- Only one row should be updated; no rows should be created or deleted
- File structure and legibility (10 points)
 - Code submitted should approximately conform to the file structure specified above
 - Code should be legible

Including node_modules with your homework submission will incur at least a 25-point penalty.

Submitting a solution that does not function will incur between a 25 and 100-point penalty.